Measuring the Cost of Persistence Guarantees in iOS Storage

CS 2640 Modern Computer Storage, Spring 2026

Valerie Chen
Harvard University

Abstract

iOS persistence APIs make writes appear instantaneous by buffering in memory and flushing asynchronously, obscur-ing both when data becomes persisted and how much a developer pays to guarantee that durability. We built an iOS benchmarking app comparing three strategies: Core Data, an append-only log over FileManager, and per-event file writes, under default and aggressive (`F_FULLFSYNC`) flush modes across seven payload sizes from 100 bytes to 50 MB. Across these 42 configurations, we discover that flush mode dominates latency. Aggressive flushing slows small Core Data writes 235x and inflates a 50 MB Core Data write 85,000x relative to default. Append-only writes pay only a 1.06–2.2x penalty, suggesting the choice of API matters far less than whether the developer ever forces a true flush. These results expose the implicit durability policies of iOS frameworks and the trade-offs they hide from app developers.

1 Context and Motivation

All modern operating systems buffer writes, i.e. a successful return from `write()` only guarantees that the kernel has accepted the bytes, not that the storage device has committed them to non-volatile media. This is a deliberate performance optimization, but it leaves a gap between the developer's perception of persistence and actual durability enforced when the application explicitly forces a flush.

On Linux and most server systems, this trade-off is well-documented. iOS is comparatively opaque: APFS is closed source, the app sandbox limits system-wide tracing, and it is standard practice among developers to call high-level frameworks like Core Data or FileManager and trust they do the right thing. As a result, roughly 1.5 billion iOS devices [2] run code where developers cannot easily determine their persistence guarantees or what enforcing them would cost.

This project asks a narrow version of that question: saving appears to be instant on iOS (in that write functions are synchronous and return immediately), but when is the data actually on disk, and what does it cost to guarantee that? We attempt to answer this through measurement since there isn't immediate clarity from Apple's own documentation, which gives only partial guidance: the `fcntl(2)` man page recommends `fcntl(fd, F_FULLFSYNC)` over plain `fsync` for true durability and notes that its cost might be high (with `F_FULLFSYNC`, "the operation may take a while to complete") [1]. Three subquestions drive the paper: how does write latency vary across **storage strategy** (high-level ORM vs. low-level append vs. naive per-event file), across **flush mode** (framework default vs. explicit aggressive flush), and across **payload size** (100 B analytics-style records to 50 MB media-style writes)?

It is useful to think of iOS persistence as having four tiers. Data lives in (1) app memory, lost on app exit; (2) the OS page cache after `write()` returns, lost on a kernel panic; (3) the SSD's internal write buffer after fsync returns, lost on power loss; and (4) on flash after `F_FULLFSYNC` returns, durable against power loss. A typical iOS developer thinks on levels 1 and 2.

iOS layers several abstractions over this ladder. **Core Data** is Apple's object-graph persistence framework that is backed by SQLite and batches writes inside an `NSManagedObjectContext` until the developer calls `.save()`. **FileManager** wraps POSIX file operations and is the primary way apps interact with APFS (Apple's proprietary file system); it does not flush on its own and requires `FileHandle.synchronize()`, `fsync(fd)`, or `fcntl(fd, F_FULLFSYNC)`. Other tools (UserDefaults, Keychain, raw SQLite, Realm, SwiftData) sit on the same underlying machinery but expose different API surfaces. We benchmark Core Data and two patterns built directly on FileManager because together they cover the spectrum from heavy ORM to bare file I/O, and are the two most common storage methods for the iOS development ecosystem.

2 Experimental Setup and Methods

2.1 Benchmark App

Our SwiftUI iOS app is organized around a `StorageBackend` protocol with three concrete implementations:

`CoreDataBackend` inserts each write as a `LogEntry` managed object into a SQLite-backed Core Data store. In default mode it calls `context.save()` every 100 inserts (a deliberately typical pattern); in aggressive mode it saves on every insert.

`AppendLogBackend` opens a single file once via `FileHandle` and appends each payload. Default mode relies on the OS to flush whereas aggressive mode calls `fcntl(fd, F_FULLFSYNC)` after every write.

`PerFileBackend` creates a new file per write under the app's `Documents/bench_scratch/` directory, modeling the pattern of one-file-per-event logging. Aggressive mode again calls `F_FULLFSYNC` during each write.

A `BenchmarkRunner` reads a configuration (backend $\times$ flush mode $\times$ payload size $\times$ trial count), sets up the chosen backend, and writes the results in JSON to the app's `Documents/results/` directory for export and analysis. The app also has visualization UI to directly view results in-app.

2.2 Measurement

Each per-write latency is measured with `clock_gettime(CLOCK_MONOTONIC_RAW)` which yields nanosecond resolution. The first 10% of writes in each run are discarded as warmup so that lazy file allocation, SQLite first-page bring-in, and other one-time effects do not bias steady-state numbers. Latencies are stored per-write, sorted, and reduced to p50, p95, and p99 percentiles. Throughput is reported as MB/s over the post-warmup window. We use percentiles instead of means as this is best practice in storage evals; tail latency is where users actually notice degradation (a single 50 ms hitch drops a frame at 60 fps).

2.3 Coverage

Our experimental matrix is 3 backends $\times$ 2 flush modes $\times$ 7 payload sizes $=$ 42 configurations, with payloads of 100 bytes, 256 bytes, 512 bytes, 1 KB (small / logging-style) and 1 MB, 10 MB, 50 MB (large / media-style). Small workloads use 1000 writes $\times$ 5 trials; large workloads use 20 writes $\times$ 3 trials. Payload bytes are randomly generated. All measurements run on a physical iPhone (17 Pro).

Workload	Strategy	Default p50	Aggressive p50	Slow.
256 bytes	Core Data	1.2 μs	282.5 μs	235$\times$
	Append Log	1.8 μs	4.0 μs	2.22$\times$
	Per-File	336 μs	6.1 ms	18$\times$
50 MB	Core Data	2.0 μs	170.5 ms	~85k$\times$
	Append Log	70 ms	74 ms	1.06$\times$
	Per-File	67 ms	74 ms	1.10$\times$

Table 1: Median per-write latency at representative small (256 bytes) and large (50 MB) payload sizes.

3 Results

Across the 42 configurations, the clearest pattern in the data is that flush mode affects latency considerably more than the choice of API. The magnitude of this effect varies substantially across backends, which appears to be driven primarily by the differing default behaviors of each framework.

3.1 Latency

Table 1 reports median latency at one small (256 bytes) and one large (50 MB) payload as representative points. At 256 bytes, both default-mode Core Data and the append log complete in approximately 1–2 μs, which is sufficiently fast to suggest that the data has not yet been flushed to disk. Under aggressive flushing, the same operations take substantially longer: the append log is roughly 2$\times$ slower, per-file is roughly 18$\times$ slower, and Core Data is roughly 235$\times$ slower. The disproportionate cost incurred by Core Data is consistent with the additional bookkeeping required to commit on every insert, though the present experiments do not isolate this effect directly.

The 50 MB row is more notable. Default Core Data reports a 2 μs p50, but this likely reflects the fact that the framework does not call `save()` during the observation window: with a batch threshold of 100 inserts, a 20-write trial does not trigger one. Consequently, the measured operation captures only the in-memory insertion, not a write to persistent storage. When `save()` is forced on every write, the same payload takes approximately 170 ms, which is consistent with the cost of actually writing 50 MB. The append log and per-file backends, by contrast, exhibit only a 1.06–1.10$\times$ difference between default and aggressive modes at this payload size. This is consistent with the interpretation that both modes already incur the cost of moving the data through the kernel, and the additional flush represents only a small marginal overhead.

3.2 Throughput

Table 2 reports throughput at the 50 MB workload. The measured values are within approximately 10% of one another across backends and flush modes. This narrow range is consistent with the device's hardware bandwidth, rather than the

Payload	Strategy	Default	Aggr.	Ratio
50 MB	Core Data	—	253 MB/s	—
	Append Log	648 MB/s	671 MB/s	0.96×
	Per-File	697 MB/s	648 MB/s	1.08×

Table 2: Sustained throughput at 50 MB writes. The Core Data default value is omitted because the framework did not call `save()` within the observation window.

API, becoming the limiting factor at this payload size, although the present measurements do not directly establish this.

4 Reflections

The most notable observation in this data is the gap between default and aggressive flush modes, particularly for Core Data. A default p50 of 2 μs on a 50 MB write does not represent a fast write to storage; rather, it indicates that the framework has deferred the write entirely. Examining only the default-mode p50 would yield the misleading conclusion that Core Data is dramatically faster than the other backends. One implication, then, is that latency alone is not a reliable indicator of whether data has actually been persisted.

Several limitations should temper the interpretation of these results. The benchmark measures the time required for `F_FULLFSYNC` to return, but does not independently verify that the data has reached non-volatile storage; doing so would require a crash-injection methodology that was outside the scope of this project. The device was not held in a controlled state: background processes, thermal conditions, and battery level may have introduced variability that this study does not quantify. All measurements were collected on a single iPhone running a single version of iOS, so the results may not generalize to other devices or operating systems. The number of trials is also modest, particularly for the large-write workloads (3 trials of 20 writes each), and individual values should therefore be regarded as indicative rather than precise.

Useful directions for future work include adding SwiftData and Realm backends to broaden the comparison, conducting a larger number of trials under more controlled device conditions, and implementing a crash-injection test to assess whether aggressive flushing achieves the durability it is intended to provide, rather than only measuring its cost.

References

[1] Apple Inc. `fcntl(2)` Mac OS X Manual Page. Apple Developer Documentation. https://developer.apple.com/library/archive/documentation/System/Conceptual/ManPages_iPhoneOS/man2/fcntl.2.html.

[2] Joe Rossignol. Apple now has 2.5 billion active devices worldwide, January 2026. https://www.macrumors.com/2026/01/29/apple-2-5-billion-active-devices/.